Data Structures & Algorithms in Python

1. What are Data Structures and Algorithms?

- **Data Structure (DS):** A way of organizing and storing data so that it can be used efficiently.
- **Algorithm:** A step-by-step set of instructions to solve a problem.
- Efficiency of an algorithm depends on **time complexity** (speed) and **space complexity** (memory usage).

---

2. Basic Python Data Structures (Built-in)

2.1 Lists

- Ordered collection of items.
- Mutable (can change after creation).
- Can contain duplicates.
- Internally implemented as **dynamic arrays**.

Operations:

- `append()` → Add at end
- `insert()` → Add at index
- `pop()` → Remove last (or at index)
- `remove()` → Remove by value

- `sort()` → Sort in-place

Example:

```
arr = [1, 2, 3]
arr.append(4)      # [1, 2, 3, 4]
arr.insert(1, 10)  # [1, 10, 2, 3, 4]
arr.pop()          # removes 4
print(arr)         # [1, 10, 2, 3]
```

2.2 Tuples

- Ordered, immutable collections.
- Faster than lists because fixed.

Example:

```
t = (10, 20, 30)
print(t[0])    # 10
```

2.3 Sets

- Unordered, unique elements only.
- Useful for **membership testing** and removing duplicates.

Example:

```
s = {1, 2, 3}
s.add(4)      # {1, 2, 3, 4}
s.remove(2)   # {1, 3, 4}
print(3 in s)  # True
```

2.4 Dictionaries

- Key-value pairs.
- Keys must be unique, values can repeat.
- Lookup is very fast (average **O(1)**).

Example:

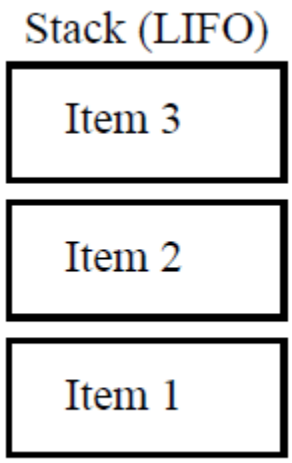
```
student = {"name": "Alice", "age": 21}
print(student["name"])  # Alice
student["grade"] = "A"
print(student)          # {'name': 'Alice', 'age': 21, 'grade': 'A'}
```

---

3. User-Defined Data Structures

3.1 Stack

- **Definition:** A linear data structure that follows **LIFO** (Last In, First Out).
- Think of a **stack of plates**: the last plate kept is the first removed.
- Operations:
  - **Push:** Add item to top.
  - **Pop:** Remove item from top.
  - **Peek/Top:** View last element.



Applications:

- Undo/Redo operations
- Backtracking (mazes, recursion)
- Expression evaluation

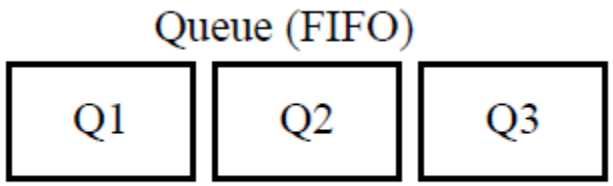
Python Example (using list):

```
stack = []
stack.append(10)  # Push
stack.append(20)
print(stack)      # [10, 20]
stack.pop()       # Pop -> removes 20
print(stack[-1])  # Peek -> 10
```

3.2 Queue

- **Definition:** A linear data structure that follows **FIFO** (First In, First Out).
- Think of a **queue at a ticket counter**: first person enters, first served.
- Operations:

- **Enqueue:** Add to end.
- **Dequeue:** Remove from front.



Applications:

- CPU scheduling
- Printers
- Breadth-First Search (BFS)

GUVI | HCL
Skill Up. Level Up

Python Example (using deque):

```
from collections import deque
q = deque()
q.append(1)   # Enqueue
q.append(2)
print(q)      # deque([1, 2])
q.popleft()   # Dequeue -> 1
print(q)      # deque([2])
```

3.3 Linked List

- **Definition:** A linear data structure where elements (nodes) are connected using pointers.
- Each **Node** has:
  - **Data**

- **Next pointer** (reference to next node)

Types:

- Singly Linked List (each node → next node)
- Doubly Linked List (node → next + prev)

Applications:

- Dynamic memory allocation
- Implementing stacks & queues

Python Example:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

# Create nodes
head = Node(1)
second = Node(2)
third = Node(3)

# Link nodes
head.next = second
second.next = third

# Traversal
```

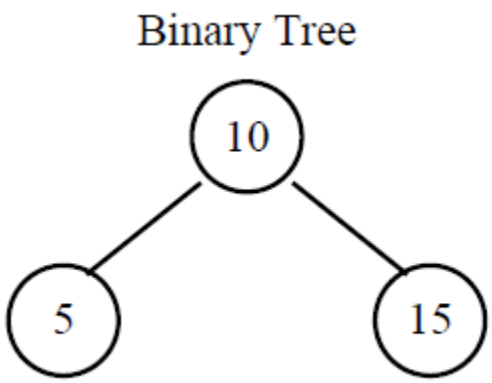
```
temp = head
while temp:
    print(temp.data, end=" -> ")
    temp = temp.next
```

3.4 Binary Tree

- **Definition:** A tree data structure where each node has at most **2 children** (left & right).

Special Types:

- Binary Search Tree (BST) → left < root < right
- Balanced Trees (AVL, Red-Black Trees)



Applications:

- Hierarchical data (filesystems)
- Searching & sorting (BST)

Python Example:

```
class Node:
    def __init__(self, key):
```

```
        self.left = None
        self.right = None
        self.key = key

def inorder(root):
    if root:
        inorder(root.left)
        print(root.key, end=" ")
        inorder(root.right)

# Create tree
root = Node(10)
root.left = Node(5)
root.right = Node(15)

inorder(root)   # Output: 5 10 15
```

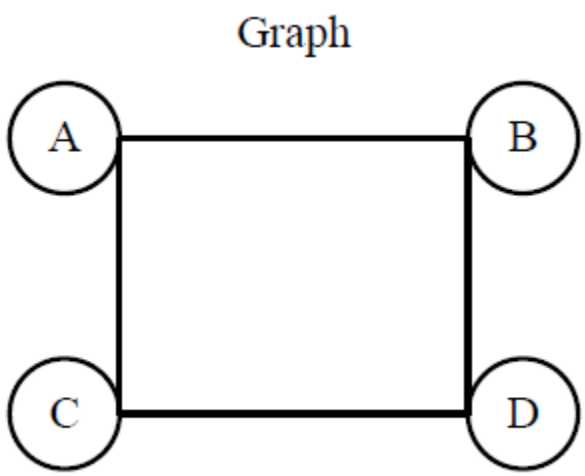
3.5 Graph

- **Definition:** A collection of **nodes (vertices)** connected by **edges**.
- Can be **directed/undirected**, **weighted/unweighted**.

Representations:

1. **Adjacency List** (efficient for sparse graphs)

2. **Adjacency Matrix** (efficient for dense graphs)



Applications:

- Social networks
- Maps & navigation (shortest path)

Python Example (Adjacency List):

```
graph = {
    "A": ["B", "C"],
    "B": ["A", "D"],
    "C": ["A", "D"],
    "D": ["B", "C"]
}
```

---

4. Algorithms

4.1 Searching

- **Linear Search (O(n))** → Check one by one
- **Binary Search (O(log n))** → Works only on sorted arrays

Binary Search Example:

```
def binary_search(arr, x):
    l, r = 0, len(arr)-1
    while l <= r:
        mid = (l+r)//2
        if arr[mid] == x:
            return mid
        elif arr[mid] < x:
            l = mid+1
        else:
            r = mid-1
    return -1
```

4.2 Sorting

- **Bubble Sort ($O(n^2)$)** → Repeatedly swap
- **Merge Sort ($O(n \log n)$)** → Divide and conquer
- **Quick Sort ($O(n \log n)$, worst $O(n^2)$)**

Bubble Sort Example:

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n-i-1):
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
    return arr
```

4.3 Graph Traversals

DFS (Depth First Search):

```
def dfs(graph, start, visited=set()):
    if start not in visited:
```

```
    print(start, end=" ")
    visited.add(start)
    for neighbor in graph[start]:
        dfs(graph, neighbor, visited)
```

BFS (Breadth First Search):

```
from collections import deque
def bfs(graph, start):
    visited, q = set(), deque([start])
    while q:
        node = q.popleft()
        if node not in visited:
            print(node, end=" ")
            visited.add(node)
            q.extend(graph[node])
```

GUVI | HCL
Skill Up. Level Up

5. Complexity Cheatsheet

Algorithm	Best	Avg	Worst
Bubble Sort	O(n)	O(n^2)	O(n^2)
Merge Sort	O(n log n)	O(n log n)	O(n log n)
Quick Sort	O(n log n)	O(n log n)	O(n^2)
Linear Search	O(1)	O(n)	O(n)
Binary Search	O(1)	O(log n)	O(log n)
DFS/BFS	O(V+E)	O(V+E)	O(V+E)

GUVI | HCL

Skill Up. Level Up

GUVI | HCL

Skill Up. Level Up